

HTML AND CSS INTRODUCTION LAB (BCS352)

B.Tech CSE - 2nd SEMESTER

SUBMITTED TO:

MR. NITISH SIR

SUBMITTED BY:

NITIN KUMAR
ID: BCS2025601

INDEX

S.No.	Name of the Experiment	Page	Date	Sign
1	Starting a Git Repository	3		
2	Pushing and Cloning a Repository	6		
3	Your First HTML Page	8		
4	Divs, Attributes and a Small Fix	11		
5	Links and Images	14		
6	Types of HTML Lists	17		
7	About Me Page	20		
8	Sign-Up Forms and Choices	24		
9	Tables with Headers and Footers	29		
10	CSS Fundamentals	33		
11	Units and CSS Variables	36		
12	Box Model, Cards and Scrolling	40		
13	Flexbox Layouts	45		
14	CSS Grid Layouts	50		
15	Responsive Pages with Media Queries	56		

EXPERIMENT NO. 1

1. Experiment Title:

Starting a Git Repository

2. Aim:

To start a Git repository for the lab folder and to save the work with commits.

3. Software / Tools Required:

Git, Visual Studio Code (terminal)

4. Theory:

Git is a version control system that records the changes made to the files of a project. git init starts a repository in a folder, git config sets the name and email that are saved in every commit, git add puts a file in the staging area and git commit saves it with a message. git status shows the state of the files and git log --oneline shows the saved commits, newest first.

5. Algorithm / Procedure:

1. Open Visual Studio Code and open the terminal in the project folder.
2. Type the Git command of the task.
3. Press Enter and read the message of Git.
4. Repeat for the next command of the task.
5. Copy the terminal output.

Task 1: Create the repository and set name and email

6. Commands:

```
git init
git config --local user.name "Nitin Kumar"
git config --local user.email "nitin.kumar@example.com"
git config user.name
git config user.email
```

7. Input:

Create a folder named html-lab on the Desktop, open it in Visual Studio Code and open the terminal. Type the commands below.

8. Output:

```
C:\Users\nikku\Desktop\html-lab> git init
Initialized empty Git repository in C:/Users/nikku/Desktop/html-lab/.git/
C:\Users\nikku\Desktop\html-lab> git config --local user.name "Nitin Kumar"
C:\Users\nikku\Desktop\html-lab> git config --local user.email "nitin.kumar@example.com"
C:\Users\nikku\Desktop\html-lab> git config user.name
Nitin Kumar
C:\Users\nikku\Desktop\html-lab> git config user.email
nitin.kumar@example.com
C:\Users\nikku\Desktop\html-lab>
```

Task 2: First commit

6. Program:

File: index.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 1 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>HTML Lab</title>
</head>
<body>
  <h1>Welcome to the HTML Lab</h1>
</body>
</html>
```

Commands:

```
git status
git add index.html
git commit -m "Add home page"
```

7. Input:

Create index.html in the html-lab folder with the code below, then type the commands.

8. Output:

```
C:\Users\nikku\Desktop\html-lab> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
C:\Users\nikku\Desktop\html-lab> git add index.html
C:\Users\nikku\Desktop\html-lab> git commit -m "Add home page"
[main (root-commit) 6d92d84] Add home page
 1 file changed, 11 insertions(+)
 create mode 100644 index.html
C:\Users\nikku\Desktop\html-lab>
```

Task 3: Second commit and git log

6. Program:

File: index.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 1 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>HTML Lab</title>
</head>
<body>
  <h1>Welcome to the HTML Lab</h1>
  <h2>Next meeting: Friday, 3 PM</h2>
</body>
</html>
```

Commands:

```
git add index.html
git commit -m "Add next meeting heading"
git log --oneline
```

7. Input:

Add the h2 heading to index.html (code below), save the file and type the commands.

8. Output:

```
C:\Users\nikku\Desktop\html-lab> git add index.html
C:\Users\nikku\Desktop\html-lab> git commit -m "Add next meeting heading"
[main ffc67a4] Add next meeting heading
 1 file changed, 1 insertion(+)
C:\Users\nikku\Desktop\html-lab> git log --oneline
ffc67a4 (HEAD -> main) Add next meeting heading
6d92d84 Add home page
C:\Users\nikku\Desktop\html-lab>
```

9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is Git?

Ans. A version control tool that saves the history of changes of a project.

Q2. What is the difference between git add and git commit?

Ans. git add stages the file, git commit saves the staged changes in the history.

Q3. What does git status show?

Ans. Which files are new, changed or staged.

Q4. Why do we set user.name and user.email?

Ans. So that every commit shows who made it.

EXPERIMENT NO. 2

1. Experiment Title:

Pushing and Cloning a Repository

2. Aim:

To send the commits to a second repository on the computer and to make a copy of it with clone.

3. Software / Tools Required:

Git, Visual Studio Code (terminal)

4. Theory:

A remote is another copy of the repository. Here it is a bare repository (a repository without working files) on the same computer. git remote add origin links it, git push sends the commits to it and git clone makes a new working copy from it. The name origin is the usual name of the main remote.

5. Algorithm / Procedure:

1. Open Visual Studio Code and open the terminal in the project folder.
2. Type the Git command of the task.
3. Press Enter and read the message of Git.
4. Repeat for the next command of the task.
5. Copy the terminal output.

Task 1: Create a bare repository and add it as origin

6. Commands:

```
git init --bare ../html-remote
git remote add origin ../html-remote
git remote -v
```

7. Input:

Open the terminal in the html-lab folder and type the commands.

8. Output:

```
C:\Users\nikku\Desktop\html-lab> git init --bare ../html-remote
Initialized empty Git repository in C:/Users/nikku/Desktop/html-remote/
C:\Users\nikku\Desktop\html-lab> git remote add origin ../html-remote
C:\Users\nikku\Desktop\html-lab> git remote -v
origin  ../html-remote (fetch)
origin  ../html-remote (push)
C:\Users\nikku\Desktop\html-lab>
```

Task 2: Push the commits

6. Commands:

```
git push origin main
```

7. Input:

Type the command in the html-lab folder.

8. Output:

```
C:\Users\nikku\Desktop\html-lab> git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 614 bytes | 307.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0
To ../html-remote
 * [new branch]      main -> main
C:\Users\nikku\Desktop\html-lab>
```

Task 3: Clone the repository

6. Commands:

```
cd ..
git clone html-remote html-copy
cd html-copy
git log --oneline
```

7. Input:

Type the commands in the terminal.

8. Output:

```
C:\Users\nikku\Desktop\html-lab> cd ..
C:\Users\nikku\Desktop> git clone html-remote html-copy
Cloning into 'html-copy'...
done.
C:\Users\nikku\Desktop> cd html-copy
C:\Users\nikku\Desktop\html-copy> git log --oneline
ffc67a4 (HEAD -> main, origin/main, origin/HEAD) Add next meeting heading
6d92d84 Add home page
C:\Users\nikku\Desktop\html-copy>
```

9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is a remote repository?

Ans. Another copy of the repository, used to share the commits.

Q2. What does git push do?

Ans. It sends the commits of the branch to the remote repository.

Q3. What is the difference between git clone and git init?

Ans. git clone copies an existing repository with its history, git init starts a new empty one.

Q4. What is a bare repository?

Ans. A repository without working files, used only for sharing.

EXPERIMENT NO. 3

1. Experiment Title:

Your First HTML Page

2. Aim:

To write a first HTML page with a title, headings and a paragraph, and to format some text.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

HTML is the language used to build web pages. Every page has a DOCTYPE, an html element, a head with the title and a body with the visible content. The headings h1 to h6 show titles from the largest to the smallest, p makes a paragraph, strong makes text bold, em makes it italic and br starts a new line inside a paragraph.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Page with a title and one heading

6. Program:

File: first.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 3 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, I am Asha</h1>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome (the browser tab title is My First Page)

Hello, I am Asha

Task 2: Headings and a paragraph

6. Program:

File: headings.html

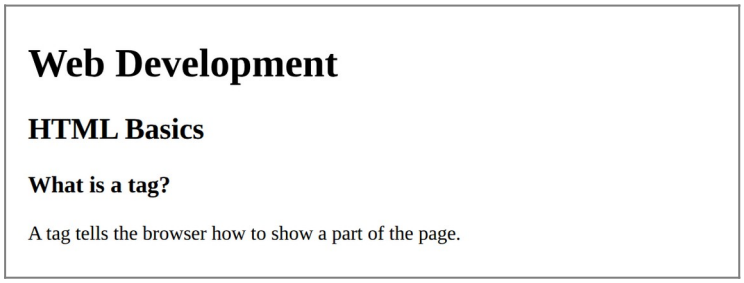
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 3 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Headings</title>
</head>
<body>
  <h1>Web Development</h1>
  <h2>HTML Basics</h2>
  <h3>What is a tag?</h3>
  <p>A tag tells the browser how to show a part of the page.</p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 3: Strong, em and line break

6. Program:

File: text-format.html

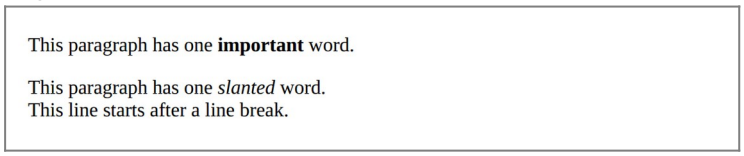
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 3 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Text Format</title>
</head>
<body>
  <p>This paragraph has one <strong>important</strong> word.</p>
  <p>This paragraph has one <em>slanted</em> word.<br>
  This line starts after a line break.</p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the use of the DOCTYPE declaration?

Ans. It tells the browser that the page is written in HTML5.

Q2. Which heading tag gives the largest text?

Ans. The h1 tag.

Q3. What is the difference between strong and em?

Ans. strong makes the text bold and em makes it italic.

Q4. Does the br tag need a closing tag?

Ans. No, br is an empty tag and has no closing tag.

EXPERIMENT NO. 4

1. Experiment Title:

Divs, Attributes and a Small Fix

2. Aim:

To group parts of a page with div elements, to add a data attribute, and to fix the errors in a broken page.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

A div groups parts of a page into a block, and a class or an id gives the block a name so that it can be found or styled later. An id must be unique, while a class can be used many times. A data attribute such as data-topic keeps extra information that is not shown on the page. Errors like an unclosed tag are fixed by closing every tag in the right order.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Three div blocks

6. Program:

File: layout.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 4 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Layout</title>
</head>
<body>
  <div class="header">This is the header.</div>
  <div class="main">This is the main part of the page.</div>
  <div class="footer">This is the footer.</div>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 2: Id and data attribute

6. Program:

File: attributes.html

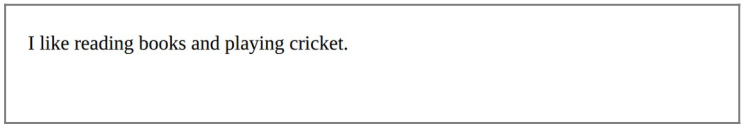
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 4 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Attributes</title>
</head>
<body>
  <div id="profile">
    <p data-topic="hobbies">I like reading books and playing cricket.</p>
  </div>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome (data-topic is not visible on the page)



Task 3: Fixing the broken page

6. Program:

File: attendance-fixed.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 4 | Task 3 -->
<html>
<head>
<title>Attendance</title>
</head>
<body>
<h1>Attendance Report</h1>
<p>Asha: 92%</p>
<p>Rohan: 85%</p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). Fixed: the h1 tag is now closed with </h1>, and the first paragraph now ends with </p>.

8. Output:

Corrected page in Chrome



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the difference between a class and an id?

Ans. An id is used for one element only, a class can be used for many elements.

Q2. What is a data attribute?

Ans. An attribute that starts with data- and stores extra information that is not shown on the page.

Q3. What does a div element do?

Ans. It groups content into a block.

Q4. What were the errors in the broken page?

Ans. The h1 tag was not closed, and the first paragraph ended with instead of .

EXPERIMENT NO. 5

1. Experiment Title:

Links and Images

2. Aim:

To link two pages together and to add a picture with alt text.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

The a tag makes a link, and its href attribute gives the page to open. Pages in the same folder are linked by using only the file name. The img tag shows a picture: src gives the file, width and height give the size, and alt gives the text that is shown if the picture is missing and that screen readers read aloud.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Home page with a link

6. Program:

File: home.html

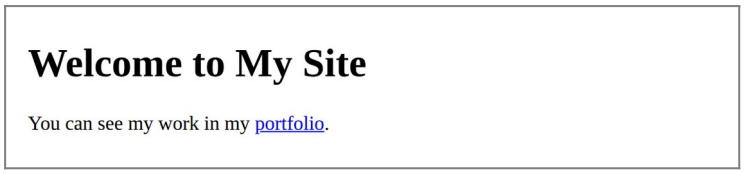
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 5 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Home</title>
</head>
<body>
  <h1>Welcome to My Site</h1>
  <p>You can see my work in my <a href="portfolio.html">portfolio</a>.</p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). Click the word portfolio.

8. Output:

home.html in Chrome



Task 2: Portfolio page with a link back

6. Program:

File: portfolio.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 5 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Portfolio</title>
</head>
<body>
  <h1>My Portfolio</h1>
  <p><a href="home.html">Go back to the home page</a></p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). Click the link.

8. Output:

portfolio.html in Chrome



Task 3: Picture with width, height and alt text

6. Program:

File: portfolio-image.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 5 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Portfolio Image</title>
</head>
<body>
  <h1>My Photo</h1>
  <img src="images/photo.jpg" width="200" height="200"
       alt="Green hills under a blue sky with a sun">
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). The picture file is images/photo.jpg; change src if your instructor gives another file name.

8. Output:

Page in Chrome



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What does the href attribute do?

Ans. It gives the address of the page that the link opens.

Q2. Why do we write alt text for an image?

Ans. It is shown when the picture cannot load and it describes the picture to screen readers.

Q3. Does the img tag need a closing tag?

Ans. No, img is an empty tag.

Q4. How do you link two pages in the same folder?

Ans. Write only the file name in href, for example portfolio.html.

EXPERIMENT NO. 6

1. Experiment Title:

Types of HTML Lists

2. Aim:

To make a bulleted list, a numbered list with a nested list, and a description list.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

The ul tag makes a bulleted list, the ol tag makes a numbered list, and li is one item of either list. A list written inside an li makes a nested list. A description list uses dl, with dt for the term and dd for its description.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Bulleted list

6. Program:

File: shopping-list.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 6 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Shopping List</title>
</head>
<body>
  <h1>Shopping List</h1>
  <ul>
    <li>Notebook</li>
    <li>Pen</li>
    <li>Bag</li>
  </ul>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 2: Numbered list with a nested list

6. Program:

File: canteen-menu.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 6 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Canteen Menu</title>
</head>
<body>
  <h1>Canteen Menu</h1>
  <ol>
    <li>Samosa</li>
    <li>Sandwich
      <ol>
        <li>Cheese</li>
        <li>Vegetable</li>
      </ol>
    </li>
  </ol>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 3: Description list

6. Program:

File: faq.html

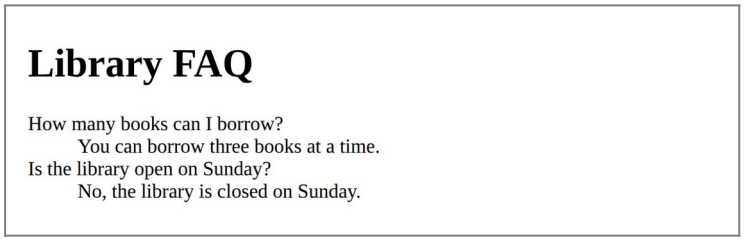
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 6 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>FAQ</title>
</head>
<body>
  <h1>Library FAQ</h1>
  <dl>
    <dt>How many books can I borrow?</dt>
    <dd>You can borrow three books at a time.</dd>
    <dt>Is the library open on Sunday?</dt>
    <dd>No, the library is closed on Sunday.</dd>
  </dl>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the difference between ul and ol?

Ans. ul shows bullets and ol shows numbers.

Q2. How do you make a nested list?

Ans. Write a new ul or ol inside an li item.

Q3. What are dt and dd?

Ans. dt is the term (question) and dd is its description (answer) inside a dl list.

Q4. Which tag is used for each item of a list?

Ans. The li tag.

EXPERIMENT NO. 7

1. Experiment Title:
About Me Page

2. Aim:
To build a short About Me page with semantic tags, an about section and a contact link.

3. Software / Tools Required:
Visual Studio Code, Google Chrome

4. Theory:
Semantic tags such as header, nav, main, section and footer describe the meaning of each part of the page. A link with href="#about" jumps to the element that has the id about on the same page. A link that starts with mailto: opens the email program with the address filled in.

5. Algorithm / Procedure:
1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Header, empty main and empty footer

6. Program:
File: about-me.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 7 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>About Me</title>
</head>
<body>
  <header>
    <h1>Nitin Kumar</h1>
    <nav>
      <a href="#about">About</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>
  <main>
  </main>
  <footer>
  </footer>
</body>
</html>
```

7. Input:
Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 2: About section inside main

6. Program:

File: about-me.html

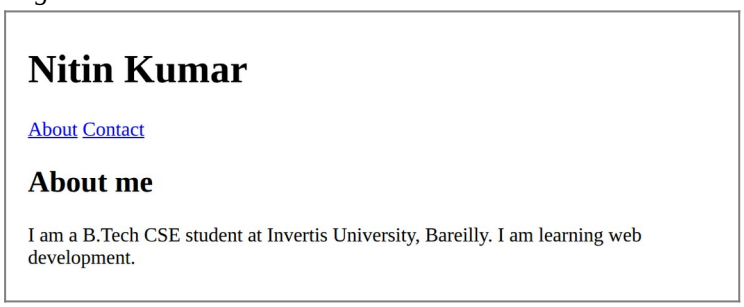
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 7 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>About Me</title>
</head>
<body>
  <header>
    <h1>Nitin Kumar</h1>
    <nav>
      <a href="#about">About</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>
  <main>
    <section id="about">
      <h2>About me</h2>
      <p>I am a B.Tech CSE student at Invertis University, Bareilly.
         I am learning web development.</p>
    </section>
  </main>
  <footer>
  </footer>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 3: Footer with an email link

6. Program:

File: about-me.html

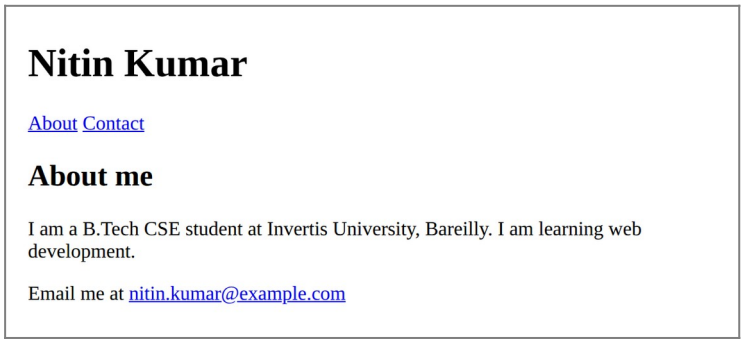
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 7 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>About Me</title>
</head>
<body>
  <header>
    <h1>Nitin Kumar</h1>
    <nav>
      <a href="#about">About</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>
  <main>
    <section id="about">
      <h2>About me</h2>
      <p>I am a B.Tech CSE student at Invertis University, Bareilly.
         I am learning web development.</p>
    </section>
  </main>
  <footer id="contact">
    <p>Email me at
      <a href="mailto:nitin.kumar@example.com">nitin.kumar@example.com</a></p>
  </footer>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). Click Contact in the top links; the page jumps to the footer.

8. Output:

Page in Chrome



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What are semantic tags? Give two examples.

Ans. Tags that describe the meaning of their content, for example header and footer.

Q2. What does href="#about" do?

Ans. It jumps to the element with the id about on the same page.

Q3. What does mailto: do in a link?

Ans. It opens the email program with the given address.

Q4. Why is the nav tag used?

Ans. It marks the part of the page that contains the navigation links.

EXPERIMENT NO. 8

1. Experiment Title:

Sign-Up Forms and Choices

2. Aim:

To make a labelled form with text, email and password fields, choices and a submit button.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

A form collects data from the user. Each input has a label, and the label is linked to the input by matching the for attribute with the id. The type password hides the typed characters. Radio buttons with the same name allow only one choice, and select makes a dropdown. The required attribute stops the form from being sent when the field is empty, and placeholder shows a hint inside the box.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Sign-up form with labelled fields

6. Program:

File: signup.html

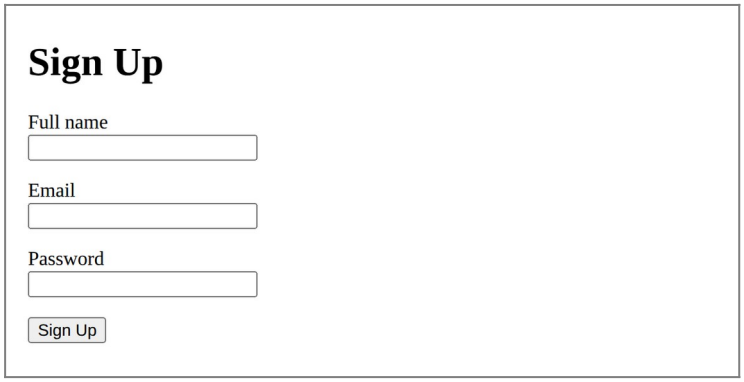
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 8 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sign Up</title>
</head>
<body>
  <h1>Sign Up</h1>
  <form method="post" action="thanks.html">
    <p>
      <label for="fullname">Full name</label><br>
      <input type="text" id="fullname" name="fullname">
    </p>
    <p>
      <label for="email">Email</label><br>
      <input type="email" id="email" name="email">
    </p>
    <p>
      <label for="password">Password</label><br>
      <input type="password" id="password" name="password">
    </p>
    <button type="submit">Sign Up</button>
  </form>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome (the password box hides the characters)



Task 2: Radio buttons and a dropdown

6. Program:

File: choices.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 8 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Choices</title>
</head>
<body>
  <h1>Choices</h1>
  <p>Gender:</p>
  <input type="radio" id="female" name="gender" value="female">
  <label for="female">Female</label>
  <input type="radio" id="male" name="gender" value="male">
  <label for="male">Male</label>
  <p>
    <label for="course">Course</label>
    <select id="course" name="course">
      <option value="bca">BCA</option>
      <option value="btech">B.Tech</option>
    </select>
  </p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 3: Required fields and a placeholder

6. Program:

File: signup-required.html

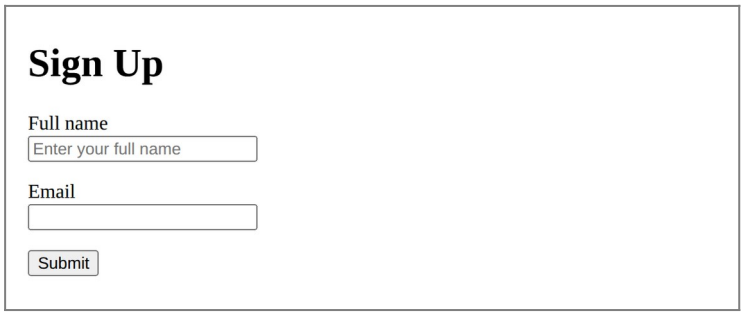
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 8 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sign Up Required</title>
</head>
<body>
  <h1>Sign Up</h1>
  <form method="post" action="thanks.html">
    <p>
      <label for="fullname">Full name</label><br>
      <input type="text" id="fullname" name="fullname"
             placeholder="Enter your full name" required>
    </p>
    <p>
      <label for="email">Email</label><br>
      <input type="email" id="email" name="email" required>
    </p>
    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). Click Submit with empty fields to see the message of the browser.

8. Output:

Page in Chrome (the placeholder shows inside the name box)



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. How is a label linked to an input?

Ans. The for attribute of the label is the same as the id of the input.

Q2. What does type="password" do?

Ans. It hides the characters that the user types.

Q3. Why do radio buttons have the same name?
Ans. So that only one of them can be selected at a time.
Q4. What does the required attribute do?
Ans. The browser shows a message and does not submit the form if the field is empty.

EXPERIMENT NO. 9

1. Experiment Title:

Tables with Headers and Footers

2. Aim:

To build a table with a caption, a header, rows and a footer, and to add scope attributes.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

A table is made with table, tr for rows, th for header cells and td for data cells. The caption gives the title of the table, and thead, tbody and tfoot divide it into header, body and footer. The scope attribute tells screen readers whether a header cell belongs to a column or to a row. The border attribute is used here only to make the cell lines visible.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Table with caption, header and three rows

6. Program:

File: marks.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 9 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Marks</title>
</head>
<body>
  <table border="1">
    <caption>Marks of Class A</caption>
    <thead>
      <tr><th>Name</th><th>Maths</th><th>Science</th></tr>
    </thead>
    <tbody>
      <tr><td>Asha</td><td>78</td><td>85</td></tr>
      <tr><td>Rohan</td><td>64</td><td>70</td></tr>
      <tr><td>Neha</td><td>91</td><td>88</td></tr>
    </tbody>
  </table>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Marks of Class A

Name	Maths	Science
Asha	78	85
Rohan	64	70
Neha	91	88

Task 2: Table with a footer row

6. Program:

File: marks-average.html

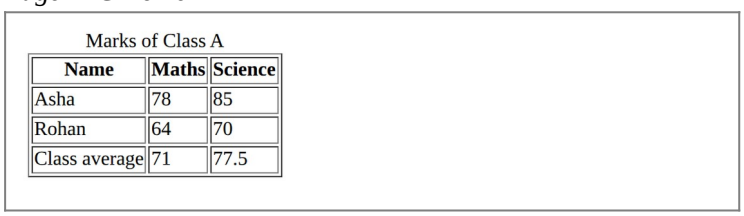
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 9 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Marks Average</title>
</head>
<body>
  <table border="1">
    <caption>Marks of Class A</caption>
    <thead>
      <tr><th>Name</th><th>Maths</th><th>Science</th></tr>
    </thead>
    <tbody>
      <tr><td>Asha</td><td>78</td><td>85</td></tr>
      <tr><td>Rohan</td><td>64</td><td>70</td></tr>
    </tbody>
    <tfoot>
      <tr><td>Class average</td><td>71</td><td>77.5</td></tr>
    </tfoot>
  </table>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). Averages: Maths (78 + 64) / 2 = 71, Science (85 + 70) / 2 = 77.5.

8. Output:

Page in Chrome



Marks of Class A

Name	Maths	Science
Asha	78	85
Rohan	64	70
Class average	71	77.5

Task 3: Table with scope attributes

6. Program:

File: marks-accessible.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 9 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Marks Accessible</title>
</head>
<body>
  <table border="1">
    <caption>Marks of Class A</caption>
    <thead>
      <tr>
        <th scope="col">Name</th>
        <th scope="col">Maths</th>
        <th scope="col">Science</th>
      </tr>
    </thead>
    <tbody>
      <tr><th scope="row">Asha</th><td>78</td><td>85</td></tr>
      <tr><th scope="row">Rohan</th><td>64</td><td>70</td></tr>
    </tbody>
  </table>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). Press F12 and check the scope attributes in the Elements panel.

8. Output:

Page in Chrome (name cells are bold because they are th cells with scope="row")

Marks of Class A

Name	Maths	Science
Asha	78	85
Rohan	64	70

9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the difference between th and td?

Ans. th is a header cell (bold and centred) and td is a normal data cell.

Q2. What is the use of thead, tbody and tfoot?

Ans. They divide the table into header, body and footer parts.

Q3. What does scope="col" mean?

Ans. The header cell is the heading of the column below it.

Q4. How was the class average of Maths found?
Ans. (78 + 64) / 2 = 71.

EXPERIMENT NO. 10

1. Experiment Title:

CSS Fundamentals

2. Aim:

To apply CSS in three ways (inline, internal and linked) and to use selectors and text styles.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

CSS is used to style HTML pages. It can be written inline in the style attribute of one element, internally inside a style tag in the head, or in a separate .css file linked with the link tag. A selector chooses the elements to style: an element selector such as h1, a class selector such as .note and an id selector such as #title. Properties such as color, font-style, text-decoration and font-size change the look of the text.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Inline and internal CSS

6. Program:

File: inline-internal.html

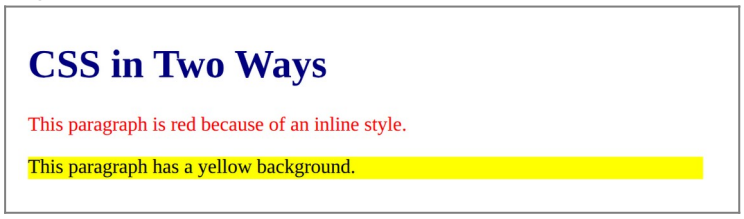
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 10 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Inline and Internal CSS</title>
  <style>
    h1 {
      color: navy;
    }
    .highlight {
      background-color: yellow;
    }
  </style>
</head>
<body>
  <h1>CSS in Two Ways</h1>
  <p style="color: red;">This paragraph is red because of an inline style.</p>
  <p class="highlight">This paragraph has a yellow background.</p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 2: Style sheet style.css

6. Program:

File: style.css

```
/* Name: Nitin Kumar | Roll: BCS2025601 | Assignment 10 | Task 2 */
h1 {
  color: navy;
}

.note {
  font-style: italic;
}

#title {
  text-decoration: underline;
}

p {
  font-size: 18px;
  color: gray;
}
```

7. Input:

Save the file and link it in an HTML page (see Task 3). The styles are shown through linked.html.

8. Output:

style.css applied through linked.html (Task 3)



Task 3: Page linked to style.css

6. Program:

File: linked.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 10 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Linked CSS</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 id="title">Linked Style Sheet</h1>
  <p class="note">This is the first note. It is styled by style.css.</p>
  <p class="note">This is the second note. It is also styled by style.css.</p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). style.css must be in the same folder as linked.html.

8. Output:

Page in Chrome



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What are the three ways of using CSS?

Ans. Inline, internal and linked (external).

Q2. What is the difference between a class selector and an id selector?

Ans. A class selector starts with a dot and can match many elements, an id selector starts with # and matches one element.

Q3. Why is a linked style sheet better?

Ans. One .css file can style many pages and the HTML stays clean.

Q4. Which property makes text italic?

Ans. font-style: italic.

EXPERIMENT NO. 11

1. Experiment Title:

Units and CSS Variables

2. Aim:

To use CSS units and to store colours and spacing in variables.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

CSS units set sizes: px is a fixed size in pixels, % is relative to the parent element and rem is relative to the font size of the root element. A CSS variable is defined with two dashes, for example --main-color, usually inside :root, and it is used with var(). Variables keep colours and spacing in one place, so a change in one line changes the whole page.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: px, % and rem units

6. Program:

File: units.html

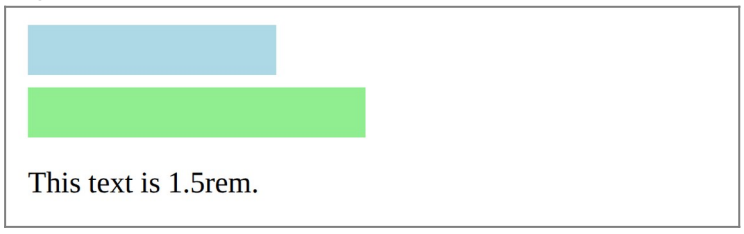
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 11 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Units</title>
  <style>
    .box1 {
      width: 200px;
      height: 40px;
      background-color: lightblue;
      margin-bottom: 10px;
    }
    .box2 {
      width: 50%;
      height: 40px;
      background-color: lightgreen;
    }
    p {
      font-size: 1.5rem;
    }
  </style>
</head>
<body>
  <div class="box1"></div>
  <div class="box2"></div>
  <p>This text is 1.5rem.</p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 2: CSS variables in the same page

6. Program:

File: variables.html

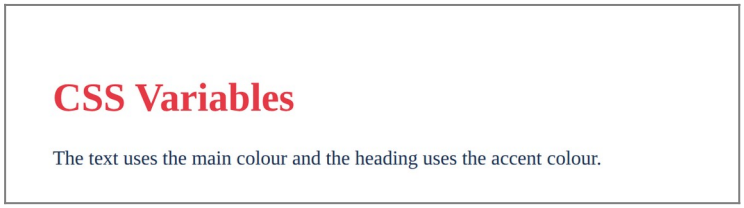
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 11 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Variables</title>
  <style>
    :root {
      --main-color: #1d3557;
      --accent-color: #e63946;
      --page-padding: 20px;
    }
    body {
      color: var(--main-color);
      padding: var(--page-padding);
    }
    h1 {
      color: var(--accent-color);
    }
  </style>
</head>
<body>
  <h1>CSS Variables</h1>
  <p>The text uses the main colour and the heading uses the accent colour.</p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 3: Variables in tokens.css

6. Program:

File: tokens.css

```
/* Name: Nitin Kumar | Roll: BCS2025601 | Assignment 11 | Task 3 */
:root {
  --primary-color: #1d3557;
  --accent-color: #e63946;
  --heading-size: 2rem;
  --spacing: 16px;
}

body {
  color: var(--primary-color);
  padding: var(--spacing);
}

h1 {
  color: var(--accent-color);
  font-size: var(--heading-size);
}
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). To see tokens.css at work, it is linked in the test page tokens-test.html.

8. Output:

tokens.css applied through tokens-test.html



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the difference between px and rem?

Ans. px is a fixed size, rem is relative to the root font size.

Q2. How do you define and use a CSS variable?

Ans. Define it as --name: value; in :root and use it as var(--name).

Q3. What does width: 50% mean?

Ans. The element is half as wide as its parent.

Q4. Why are CSS variables useful?

Ans. A colour or spacing is written once and can be changed in one place.

EXPERIMENT NO. 12

1. Experiment Title:

Box Model, Cards and Scrolling

2. Aim:

To compare the box sizing types, to style a card and to make a box scroll.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

Every element is a box made of content, padding, border and margin. With box-sizing: content-box the width covers only the content, so padding and border make the box wider. With border-box the width includes the padding and the border. The border-radius property rounds the corners, and overflow: auto adds a scroll bar when the content is bigger than the box.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: content-box and border-box

6. Program:

File: box-model.html

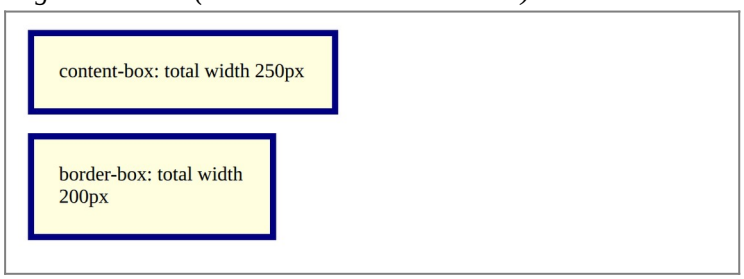
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 12 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Box Model</title>
  <style>
    .box {
      width: 200px;
      padding: 20px;
      border: 5px solid navy;
      background-color: lightyellow;
      margin-bottom: 15px;
    }
    .first {
      box-sizing: content-box;
    }
    .second {
      box-sizing: border-box;
    }
  </style>
</head>
<body>
  <div class="box first">content-box: total width 250px</div>
  <div class="box second">border-box: total width 200px</div>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome (the content-box box is wider)



Task 2: A card

6. Program:

File: card.html

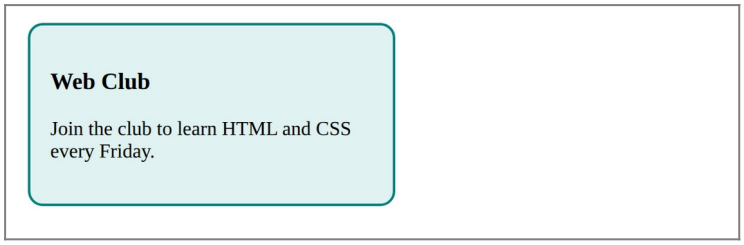
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 12 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Card</title>
  <style>
    .card {
      background-color: #e0f2f1;
      border: 2px solid teal;
      border-radius: 12px;
      padding: 16px;
      width: 260px;
    }
  </style>
</head>
<body>
  <div class="card">
    <h3>Web Club</h3>
    <p>Join the club to learn HTML and CSS every Friday.</p>
  </div>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 3: A scrolling box

6. Program:

File: scroll.html

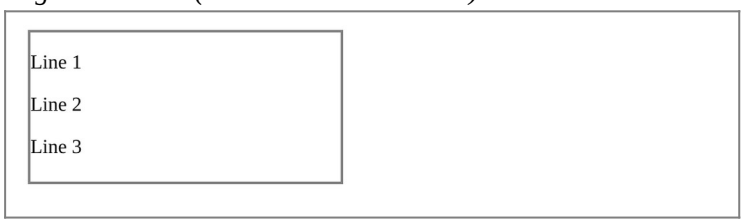
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 12 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Scroll</title>
  <style>
    .box {
      width: 250px;
      height: 120px;
      overflow: auto;
      border: 2px solid gray;
    }
  </style>
</head>
<body>
  <div class="box">
    <p>Line 1</p>
    <p>Line 2</p>
    <p>Line 3</p>
    <p>Line 4</p>
    <p>Line 5</p>
  </div>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome (the box has a scroll bar)



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What are the four parts of the box model?

Ans. Content, padding, border and margin.

Q2. What is the difference between content-box and border-box?

Ans. In content-box padding and border are added to the width, in border-box they are inside the width.

Q3. What does border-radius do?
Ans. It makes the corners of a box round.
Q4. What does overflow: auto do?
Ans. It shows a scroll bar only when the content does not fit in the box.

EXPERIMENT NO. 13

1. Experiment Title:

Flexbox Layouts

2. Aim:

To lay out a navigation bar, centred cards and cards that wrap, using Flexbox.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

Flexbox arranges items in a row or in a column. display: flex on the parent makes its children flex items. justify-content aligns the items along the main axis (left to right in a row) and align-items aligns them across it. With flex-wrap: wrap the items move to the next row when there is no space left in the row.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Navigation bar

6. Program:

File: nav-flex.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 13 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Navigation Bar</title>
  <style>
    body {
      margin: 0;
    }
    nav {
      display: flex;
      gap: 24px;
      background-color: #222;
      padding: 14px 20px;
    }
    nav a {
      color: white;
      text-decoration: none;
    }
  </style>
</head>
<body>
  <nav>
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Events</a>
    <a href="#">Contact</a>
  </nav>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 2: Centred cards

6. Program:

File: centred-cards.html

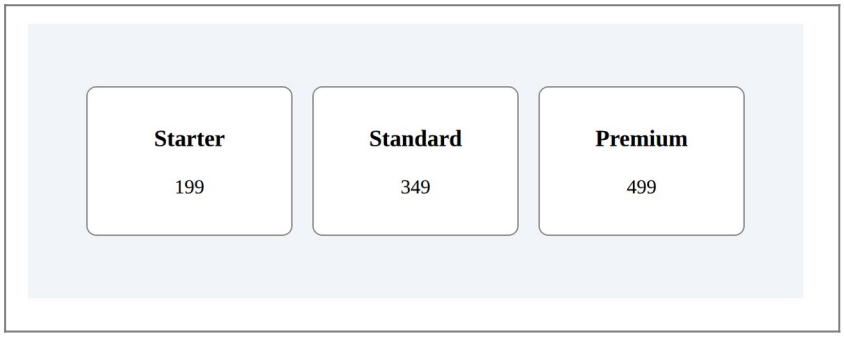
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 13 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Centred Cards</title>
  <style>
    .row {
      display: flex;
      justify-content: center;
      align-items: center;
      gap: 16px;
      height: 220px;
      background-color: #f1f5f9;
    }
    .card {
      width: 140px;
      padding: 12px;
      text-align: center;
      background-color: white;
      border: 1px solid gray;
      border-radius: 8px;
    }
  </style>
</head>
<body>
  <div class="row">
    <div class="card"><h3>Starter</h3><p>199</p></div>
    <div class="card"><h3>Standard</h3><p>349</p></div>
    <div class="card"><h3>Premium</h3><p>499</p></div>
  </div>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 3: Wrapping cards

6. Program:

File: wrapping-cards.html

```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 13 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Wrapping Cards</title>
  <style>
    .row {
      display: flex;
      flex-wrap: wrap;
      gap: 12px;
    }
    .card {
      width: 130px;
      padding: 10px;
      text-align: center;
      border: 1px solid gray;
      border-radius: 8px;
      background-color: #fff7e6;
    }
  </style>
</head>
<body>
  <h1>Stationery Shop</h1>
  <div class="row">
    <div class="card"><h3>Notebook</h3><p>Price 60</p></div>
    <div class="card"><h3>Pen</h3><p>Price 15</p></div>
    <div class="card"><h3>Bag</h3><p>Price 350</p></div>
    <div class="card"><h3>Book</h3><p>Price 120</p></div>
  </div>
</body>
</html>
```

7. Input:

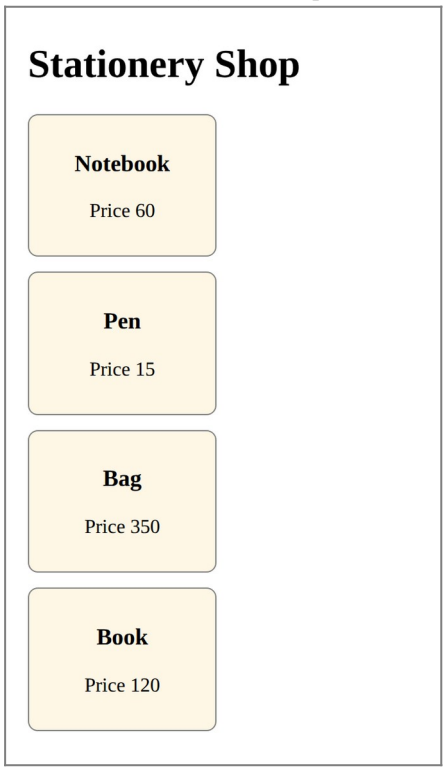
Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). Make the browser window narrow to see the cards wrap.

8. Output:

Wide window



Narrow window (cards wrap onto new rows)



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What does display: flex do?

Ans. It makes the children of the element flex items placed in a row.

Q2. What is the difference between justify-content and align-items?

Ans. justify-content aligns along the main axis, align-items aligns across it.

Q3. What does flex-wrap: wrap do?

Ans. Items move to a new row when the row is full.

Q4. How do you place items in the centre of a flex row?

Ans. Use justify-content: center and align-items: center.

EXPERIMENT NO. 14

1. Experiment Title:

CSS Grid Layouts

2. Aim:

To make a grid with equal columns, a responsive gallery and a layout with named areas.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

CSS Grid arranges items in rows and columns. display: grid with grid-template-columns defines the columns, the fr unit shares the free space equally and gap sets the space between the items. repeat(auto-fill, minmax(150px, 1fr)) makes a gallery in which the number of columns changes with the window width. With grid-template-areas each part of the page is placed by name.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Three equal columns

6. Program:

File: grid-first.html

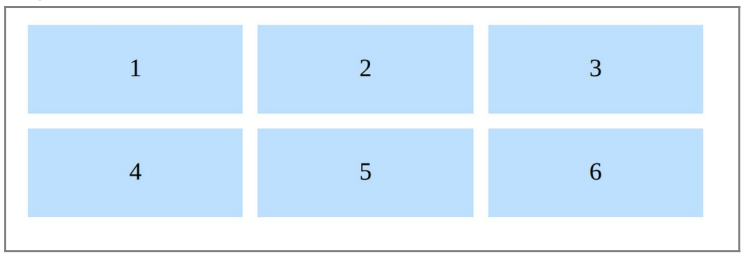
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 14 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>First Grid</title>
  <style>
    .grid {
      display: grid;
      grid-template-columns: 1fr 1fr 1fr;
      gap: 12px;
    }
    .grid div {
      background-color: #bde0fe;
      padding: 24px;
      text-align: center;
      font-size: 20px;
    }
  </style>
</head>
<body>
  <div class="grid">
    <div>1</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
    <div>6</div>
  </div>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



Task 2: Responsive photo gallery

6. Program:

File: gallery.html

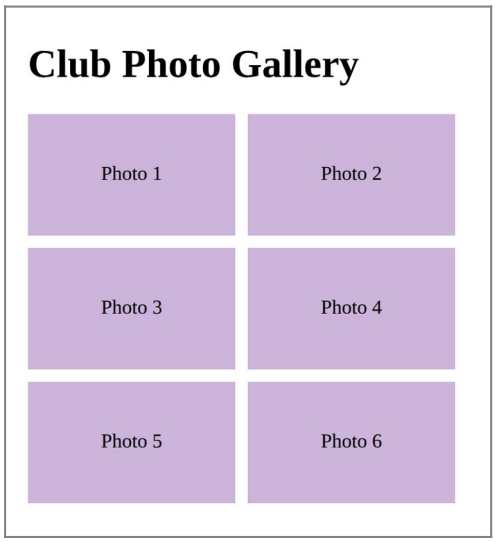
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 14 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Gallery</title>
  <style>
    .gallery {
      display: grid;
      grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
      gap: 10px;
    }
    .gallery div {
      background-color: #cdb4db;
      padding: 40px 10px;
      text-align: center;
    }
  </style>
</head>
<body>
  <h1>Club Photo Gallery</h1>
  <div class="gallery">
    <div>Photo 1</div>
    <div>Photo 2</div>
    <div>Photo 3</div>
    <div>Photo 4</div>
    <div>Photo 5</div>
    <div>Photo 6</div>
  </div>
</body>
</html>
```

7. Input:

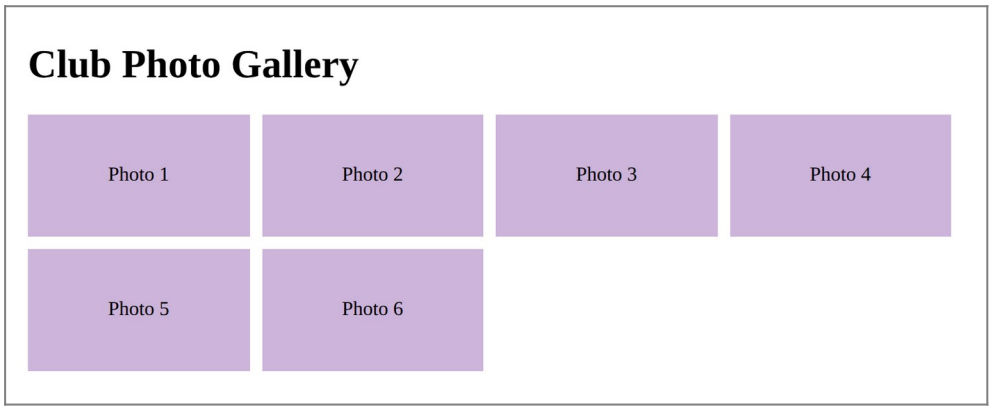
Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). Change the window width to see the number of columns change.

8. Output:

Narrow window



Wide window



Task 3: Layout with named areas

6. Program:

File: dashboard-areas.html

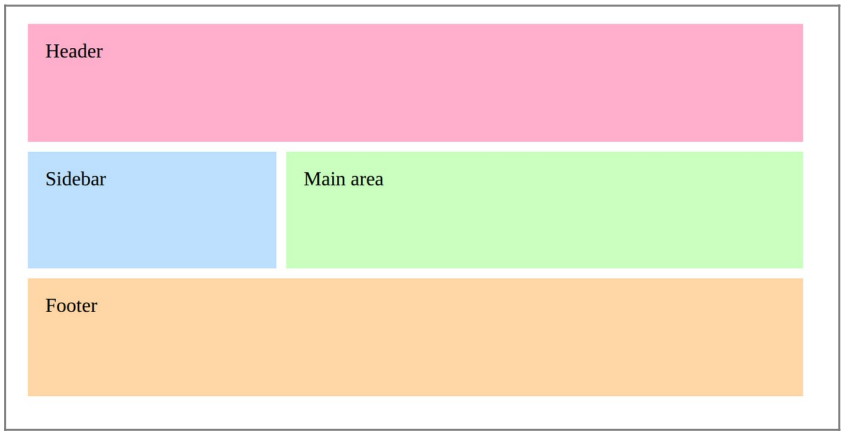
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 14 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dashboard</title>
  <style>
    .layout {
      display: grid;
      grid-template-columns: 200px 1fr;
      grid-template-areas:
        "header header"
        "sidebar main"
        "footer footer";
      gap: 8px;
      min-height: 300px;
    }
    .header  { grid-area: header;  background-color: #ffafcc; padding: 14px; }
    .sidebar { grid-area: sidebar; background-color: #bde0fe; padding: 14px; }
    .main    { grid-area: main;    background-color: #caffbf; padding: 14px; }
    .footer  { grid-area: footer;  background-color: #ffd6a5; padding: 14px; }
  </style>
</head>
<body>
  <div class="layout">
    <div class="header">Header</div>
    <div class="sidebar">Sidebar</div>
    <div class="main">Main area</div>
    <div class="footer">Footer</div>
  </div>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Page in Chrome



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What does the fr unit mean?

Ans. It is one share of the free space of the grid.

Q2. What does repeat(auto-fill, minmax(150px, 1fr)) do?

Ans. It fits as many columns of at least 150px as possible in a row.

Q3. What is the use of grid-template-areas?

Ans. It places the parts of the page by name, such as header, sidebar and main.

Q4. What is the difference between Flexbox and Grid?

Ans. Flexbox works in one direction (row or column), Grid works in rows and columns together.

EXPERIMENT NO. 15

1. Experiment Title:

Responsive Pages with Media Queries

2. Aim:

To use a media query to change a layout, to make a picture scale with its box and to set text sizes with rem units.

3. Software / Tools Required:

Visual Studio Code, Google Chrome

4. Theory:

A responsive page changes with the size of the screen. The viewport meta tag makes the page fit the width of a phone. In the mobile-first method the base styles are written for phones and a media query such as @media (min-width: 600px) adds the styles for wider screens. max-width: 100% with height: auto makes a picture shrink with its box, and rem units scale the text from the root font size.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .html extension (and a .css file where used).
3. Write the code and save the file.
4. Open the .html file in Google Chrome.
5. Check the output.

Task 1: Cards that stack on phones

6. Program:

File: mobile-first.html

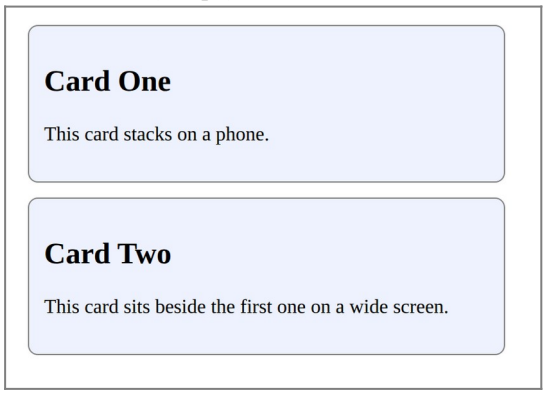
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 15 | Task 1 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Mobile First</title>
  <style>
    .card {
      border: 1px solid gray;
      border-radius: 8px;
      padding: 12px;
      margin-bottom: 12px;
      background-color: #eef2ff;
    }
    @media (min-width: 600px) {
      .cards {
        display: flex;
        gap: 12px;
      }
      .card {
        flex: 1;
        margin-bottom: 0;
      }
    }
  </style>
</head>
<body>
  <div class="cards">
    <div class="card">
      <h2>Card One</h2>
      <p>This card stacks on a phone.</p>
    </div>
    <div class="card">
      <h2>Card Two</h2>
      <p>This card sits beside the first one on a wide screen.</p>
    </div>
  </div>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). Make the window narrower and wider than 600px.

8. Output:

Phone width (400px): cards stack



Wide window (800px): cards side by side



Task 2: Picture that scales

6. Program:

File: responsive-image.html

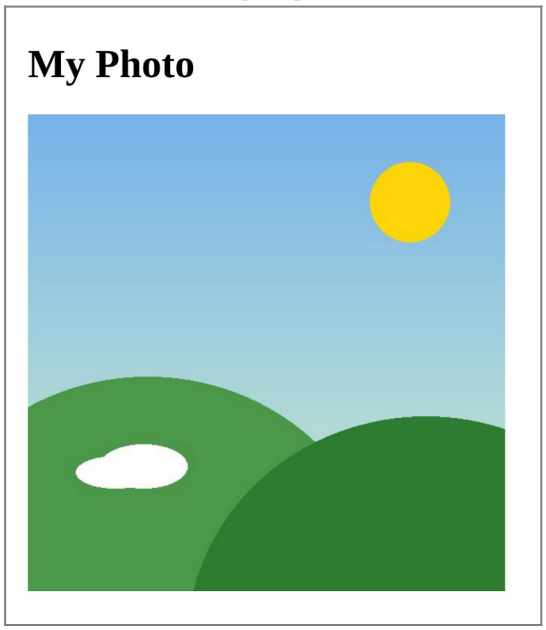
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 15 | Task 2 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Responsive Image</title>
  <style>
    img {
      max-width: 100%;
      height: auto;
    }
  </style>
</head>
<body>
  <h1>My Photo</h1>
  <img src="images/photo.jpg" alt="Green hills under a blue sky with a sun">
</body>
</html>
```

7. Input:

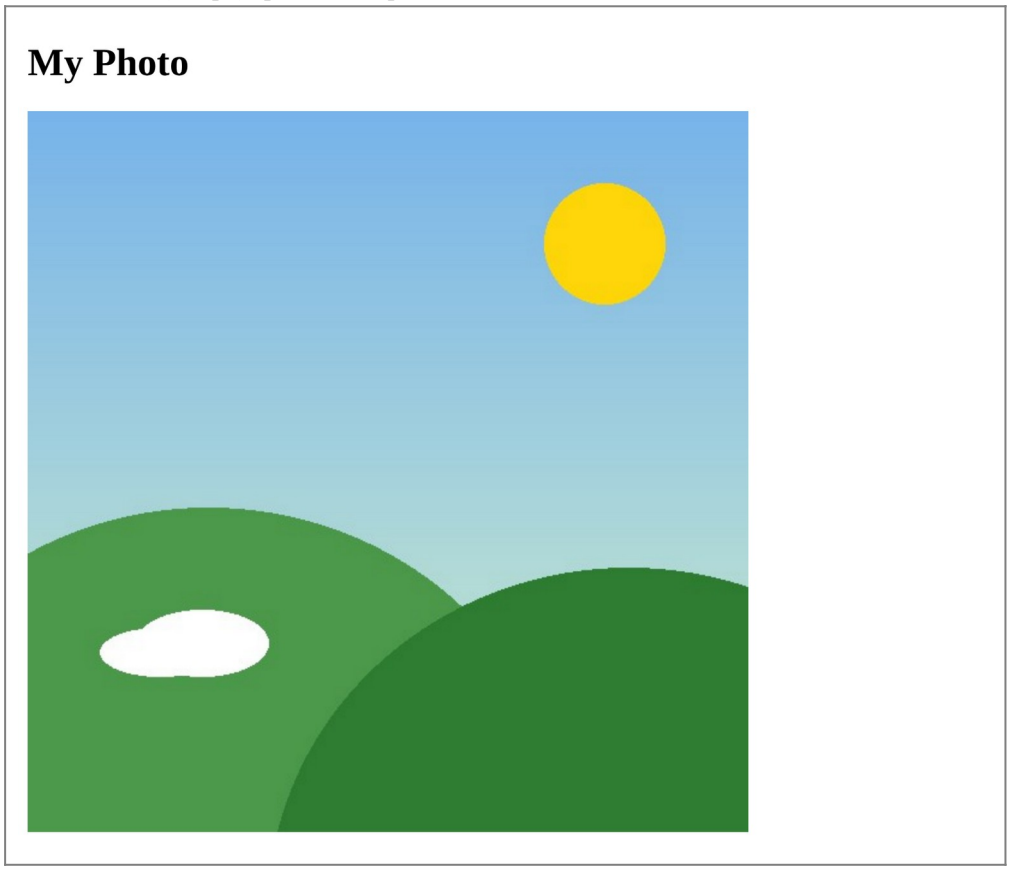
Open the HTML file in Chrome (right-click the file, Open with, Google Chrome). The picture file is images/photo.jpg; change src if your instructor gives another file name.

8. Output:

Narrow window (400px): picture shrinks



Wide window (800px): picture keeps its own size



Task 3: Text sizes with rem

6. Program:

File: responsive-text.html

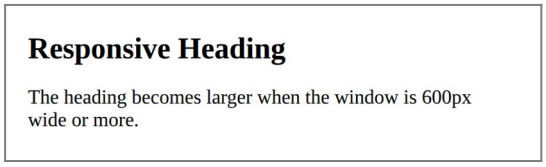
```
<!DOCTYPE html>
<!-- Name: Nitin Kumar | Roll: BCS2025601 | Assignment 15 | Task 3 -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Responsive Text</title>
  <style>
    body {
      font-size: 1rem;
    }
    h1 {
      font-size: 1.5rem;
    }
    @media (min-width: 600px) {
      h1 {
        font-size: 2.5rem;
      }
    }
  </style>
</head>
<body>
  <h1>Responsive Heading</h1>
  <p>The heading becomes larger when the window is 600px wide or more.</p>
</body>
</html>
```

7. Input:

Open the HTML file in Chrome (right-click the file, Open with, Google Chrome).

8. Output:

Narrow window (400px)



Wide window (800px)



9. Result:

The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the use of the viewport meta tag?

Ans. It makes the page width equal to the device width, so the page fits a phone.

Q2. What is a media query?

Ans. A rule that applies CSS only when a condition, such as a minimum screen width, is true.

Q3. What does mobile-first mean?

Ans. The base styles are written for phones and media queries add styles for larger screens.

Q4. Why do we use max-width: 100% for pictures?

Ans. So that the picture never becomes wider than its box.